

RAPPORT DE STAGE II

Filière : **Ingénierie Informatique et Technologies Emergentes (2ITE)**
3^{ème} année Cycle Ingénieur

Déploiement Automatisé d'une Infrastructure Moderne et Sécurisée sur AWS avec Terraform (IaC)

Réalisé à : ToumAI
(du 03 Juin 2025 à 03 Aout 2025)

Réalisé par :

Sifeddine Douidy

Encadré par :

Tech Lead. Yasser Janah

Année universitaire : 2025/2026

Dédicaces

Je tiens à dédier ce rapport à mes chers parents, qui ont toujours été présents pour moi et m'ont soutenu tout au long de ce stage. Votre amour inconditionnel, votre soutien et vos encouragements ont été essentiels pour ma réussite. Je vous remercie pour tous les efforts que vous avez faits pour moi.

À mes amis et collègues. Votre amitié précieuse, vos conseils avisés et votre présence constante ont été une source de réconfort et de joie pendant ce stage. Merci d'avoir été là pour moi, d'avoir partagé les hauts et les bas, et d'avoir rendu cette expérience encore plus spéciale.

Remerciements

Je profite de l'occasion de ce rapport pour exprimer mes sincères remerciements. Tout d'abord, je remercie le Tout-Puissant **ALLAH** de m'avoir accordé la patience, le courage et la capacité de mener à bien ce stage.

En second lieu, je tiens à exprimer ma gratitude envers mon encadrant Mr. **JANAH Yasser**, Tech Lead à l'entreprise **ToumAI**, pour ses efforts, son soutien, ses encouragements et ses conseils précieux tout au long de cette expérience.

Je tiens à exprimer ma sincère gratitude à l'ensemble du corps professoral et administratif de l'Ecole Nationale des Sciences Appliquées de El Jadida (ENSAJ), et plus particulièrement au département des Télécommunications, Réseaux et Informatique (TRI). Leur enseignement de qualité et leur dévouement ont grandement contribué à ma formation et à mon apprentissage. Leur expertise et leurs efforts continus pour offrir une éducation de haut niveau sont hautement appréciés.

Je souhaite remercier du fond du cœur ma chère famille et mes chers amis, qui par leur soutien, leurs prières et leurs encouragements, m'ont permis de surmonter tous les obstacles sur mon chemin.

Merci à toutes et à tous.

Liste des acronymes du projet :

- **AWS** - Amazon Web Services
- **VPC** - Virtual Private Cloud
- **ECS** - Elastic Container Service
- **ALB** - Application Load Balancer
- **RDS** - Relational Database Service
- **IAM** - Identity and Access Management
- **WAF** - Web Application Firewall
- **IaC** - Infrastructure as Code
- **CI/CD** - Continuous Integration / Continuous Deployment (ou Delivery)
- **API** - Application Programming Interface
- **DNS** - Domain Name System
- **CIDR** - Classless Inter-Domain Routing
- **NAT** - Network Address Translation
- **AZ** - Availability Zone
- **JSON** - JavaScript Object Notation
- **YAML** - Yet Another Markup Language
- **SQL** - Structured Query Language

Résumé

Ce rapport a été rédigé dans le cadre du projet de stage d'été. Il détaille la conception et la mise en œuvre d'une **infrastructure cloud robuste et automatisée** sur **Amazon Web Services (AWS)**, destinée à héberger des applications web modernes et évolutives, telles que des systèmes ERP et CRM.

Dans un contexte de développement agile, la mise en place d'une infrastructure fiable, sécurisée et reproductible est un prérequis fondamental. Ce projet répond à ce besoin en créant une **fondation cloud** qui isole les environnements de développement, de test et de production. L'architecture s'appuie sur des réseaux privés virtuels (**VPC**) avec des sous-réseaux publics et privés pour sécuriser les services, tels que les bases de données (**RDS**) et les conteneurs applicatifs (**ECS**).

Pour réaliser cette solution, nous avons adopté une approche d'Infrastructure as Code (**IaC**) en utilisant **Terraform** pour définir et gérer l'ensemble des ressources cloud. Le processus de déploiement est entièrement automatisé via une pipeline de CI/CD avec **GitHub Actions**, garantissant la validation et la planification cohérente des changements. Cette infrastructure est conçue pour supporter une architecture de microservices, capable d'exécuter des applications conteneurisées développées avec des frameworks comme **Next.js** et **Spring Boot**.

Abstract

This report was written as part of a summer internship project. It details the design and implementation on **Amazon Web Services (AWS)** of a robust and automated **cloud infrastructure**, serving as a foundation to host modern and scalable web applications, such as ERP and CRM systems. In the context of agile development, establishing a reliable, secure, and reproducible infrastructure is a fundamental prerequisite. This project addresses this need by creating a **cloud foundation** that isolates development, testing, and production environments. The architecture relies on Virtual Private Clouds (**VPC**) with public and private subnets to secure services, such as databases (**RDS**) and application containers (**ECS**). To build this solution, we adopted an Infrastructure as Code (**IaC**) approach using **Terraform** to define and manage all cloud resources. The deployment process is fully automated through a CI/CD pipeline with **GitHub Actions**, ensuring the consistent validation and planning of changes. This infrastructure is designed to support a microservices architecture, capable of running containerized applications developed with frameworks like **Next.js** and **Spring Boot**.

Table des matières

Table des matières

Table des figures

Liste des tableaux

Introduction générale

1	Contexte Général	1
1.1	Introduction	1
1.2	Organisme d'accueil	1
1.3	Contexte général du projet	2
1.3.1	Définitions générales	2
1.3.2	Problématique	2
1.3.3	Solution proposée	2
1.4	Gestion du projet	2
1.4.1	Choix de la méthode de gestion du projet	2
1.4.2	Planification et déroulement	3
1.5	Conclusion	4
2	Revue de Littérature : Fondements Théoriques	5
2.1	Introduction	5
2.2	L'Infrastructure as Code (IaC)	5
2.2.1	Approche Déclarative vs. Impérative	5
2.3	La Philosophie DevOps	6
2.4	Les Principes du Cloud-Native	7
2.5	Architecture Applicative Cible : Les Microservices	8
2.5.1	Réponse aux besoins des microservices	8
2.6	Conclusion	9
3	Spécification Des Exigences	10
3.1	Introduction	10
3.2	Étude des exigences	10
3.2.1	Objectifs du Projet	10
3.2.2	Besoins fonctionnels (Capacités de la plateforme)	10
3.2.3	Besoins non fonctionnels (Attributs de qualité de la plateforme)	11
3.3	Exigences Techniques	12
3.4	Analyse de l'existant et des solutions alternatives	13
3.5	Conclusion	15
4	Analyse et Conception	16

4.1	Introduction	16
4.2	Méthode de modélisation	16
4.3	Identification des acteurs de la plateforme	17
4.4	Conception de l'Architecture	18
4.4.1	Diagramme d'Architecture Logique	18
4.4.2	Diagramme d'Architecture Physique	19
4.5	Conclusion	19
5	Réalisation Technique	20
5.1	Introduction	20
5.2	Principes de Fonctionnement de Terraform	20
5.3	Environnement de développement technique	21
5.4	Environnement de développement logiciel	22
5.5	Mise en Œuvre et Démonstration de la Plateforme	22
5.5.1	Structure du Code et Gestion des Environnements	22
5.6	Démonstration du Workflow de Déploiement	24
5.6.1	Étape 1 : Déclaration du Nouveau Service	24
5.6.2	Étape 2 : Déclenchement de la Pipeline CI/CD	25
5.6.3	Étape 3 : Validation et Planification Automatisées	25
5.7	Conclusion	26
	Conclusion Générale et Perspectives	27
	Webographie	28

Table des figures

Figure 1.1 :	ToumAI Logo	1
Figure 1.2 :	Le processus de Scrum	3
Figure 1.3 :	Diagramme de Gantt du projet	4
Figure 2.1 :	Approche Impérative	6
Figure 2.2 :	Approche Déclarative	6
Figure 2.3 :	Schéma de DevOps	7
Figure 2.4 :	Principes du cloud native	7
Figure 2.5 :	L'architecture de microservices	8
Figure 3.1 :	Logo d'AWS	12
Figure 3.2 :	Logo de Terraform	13
Figure 3.3 :	Logo d'Infracost	13
Figure 3.4 :	Logo de GitHub Actions	13
Figure 3.5 :	AWS Elastic Beanstalk Logo	14
Figure 3.6 :	Terraform Cloud Logo	14
Figure 3.7 :	AWS CloudFormation Logo	15
Figure 4.1 :	Cycle de modélisation et de livraison continue avec IaC et DevOps	17
Figure 4.2 :	Diagramme d'Architecture Logique	18
Figure 4.3 :	Diagramme d'Architecture Physique	19
Figure 5.1 :	Architecture interne de Terraform	21
Figure 5.2 :	Dossier Modules	23
Figure 5.3 :	Dossier Live	23
Figure 5.4 :	Organisation des dossiers Modules et Live	23
Figure 5.5 :	Les environnements existants définis via les Workspaces Terraform	23
Figure 5.6 :	Avant l'ajout du service notifications	25
Figure 5.7 :	Après l'ajout du service notifications	25
Figure 5.8 :	Modification du fichier develop.tfvars pour intégrer un nouveau service	25
Figure 5.9 :	Déclenchement automatique de la pipeline CI/CD sur GitHub	25
Figure 5.10 :	Résultat du terraform plan dans GitHub Actions	26
Figure 5.11 :	Estimation des coûts générée par Infracost	26
Figure 5.12 :	Planification Terraform et estimation des coûts via Infracost	26

Liste des tableaux

Tableau 4.1 : Acteurs de la plateforme et leurs rôles 17

Introduction générale

Dans le monde du développement logiciel moderne, la vitesse et la fiabilité du déploiement des applications sont devenues des facteurs de compétitivité essentiels. Cependant, les processus de gestion d'infrastructure traditionnels, souvent manuels et spécifiques à chaque projet, constituent un frein majeur à l'innovation. Ils sont non seulement lents et sujets aux erreurs, mais créent également des incohérences critiques entre les environnements de développement et de production.

Pour répondre à ces défis, ce projet de stage d'été a porté sur la conception et la réalisation d'un **système de déploiement cloud interne** sur AWS. L'objectif est de remplacer les approches manuelles par une fondation d'infrastructure centralisée, automatisée et reproductible. En adoptant les principes de l'Infrastructure as Code (IaC), nous avons créé un mécanisme qui permet aux équipes de développement de provisionner et de gérer leurs applications, comme des systèmes ERP/CRM, de manière autonome et sécurisée.

Ce rapport détaille les étapes de la création de cette **solution**, structuré en cinq chapitres. Le premier chapitre introduit le contexte général du projet, en décrivant la problématique de la gestion d'infrastructure et les objectifs de l'automatisation. Le deuxième chapitre présente la revue de littérature. Il décrit les fondements théoriques du projet, notamment l'Infrastructure as Code (IaC), la philosophie DevOps, les principes du Cloud-Native ainsi que l'architecture microservices, qui constitue le modèle applicatif cible. Le troisième chapitre présente le cahier des charges, en mettant en lumière les exigences fonctionnelles (capacités du système) et non fonctionnelles (sécurité, haute disponibilité). Le quatrième chapitre se concentre sur la conception de l'architecture, en utilisant des diagrammes logiques et physiques pour modéliser la solution. Enfin, le cinquième chapitre détaille les technologies et outils clés (AWS, Terraform, GitHub Actions) qui ont permis la réalisation de cette infrastructure.

Contexte Général

1.1 Introduction

Ce chapitre présente la démarche de notre projet : la conception et la réalisation d'un système de déploiement automatisé sur AWS. Pour répondre aux défis des processus manuels, souvent lents et risqués, nous avons mis en œuvre une architecture entièrement gérée par le code (Infrastructure as Code). En utilisant **Terraform**, cette infrastructure devient reproductible et facile à maintenir. De plus, une pipeline de CI/CD, orchestrée par **GitHub Actions**, a été intégrée pour assurer des mises en production rapides, sécurisées et cohérentes.

1.2 Organisme d'accueil

ToumAI, basée au Maroc, se spécialise dans l'intelligence artificielle appliquée à la voix et au langage. Elle propose des solutions vocales multilingues permettant aux entreprises de mieux comprendre et interagir avec leurs clients.

Grâce à sa technologie innovante, ToumAI aide à analyser les conversations en temps réel, détecter les émotions et optimiser l'expérience client. Soutenue par des partenaires comme **Orange Ventures**, elle s'impose comme un acteur clé de l'IA inclusive et accessible.



Figure 1.1. ToumAI Logo

1.3 Contexte général du projet

1.3.1 Définitions générales

1.3.2 Problématique

Dans un environnement de développement logiciel rapide, les processus de gestion d'infrastructure manuels posent des défis majeurs. La création et la configuration d'environnements cloud (développement, test, production) en cliquant dans une console web sont lentes, sujettes aux erreurs humaines et difficiles à reproduire de manière identique. Cette approche traditionnelle entraîne des incohérences entre les environnements ("ça marche sur ma machine, mais pas en production"), des failles de sécurité dues à des configurations divergentes, et une absence de traçabilité ou de contrôle de version pour l'infrastructure.

1.3.3 Solution proposée

Pour résoudre cette problématique, ce projet met en œuvre un écosystème cloud entièrement automatisé en adoptant une approche d'Infrastructure as Code (IaC). La solution s'articule autour de **Terraform**, qui nous permet de définir l'ensemble de l'architecture AWS (réseaux, bases de données, conteneurs) dans des fichiers de code versionnés. Cette méthode garantit que chaque environnement est créé de manière cohérente et reproductible. De plus, une pipeline de CI/CD avec **GitHub Actions** a été développée pour automatiser la validation et la planification des changements, assurant ainsi des déploiements rapides, fiables et sécurisés.

1.4 Gestion du projet

1.4.1 Choix de la méthode de gestion du projet

Afin de pouvoir comprendre les tâches à réaliser, nous adoptons une des méthodes de gestion de projet les plus populaires, qui est la méthode Scrum.

Scrum est un cadre de travail collaboratif visant à obtenir une amélioration continue par le biais de la livraison itérative et incrémentale de produits destinés à satisfaire les clients. Basé sur la théorie du contrôle empirique des processus, Scrum met l'accent sur l'adaptabilité et la transparence dans la gestion de projets complexes.

L'objectif principal de Scrum est de répondre aux besoins du client en mettant en place un environnement de communication transparent, de responsabilité collective et de progrès continu. Le processus commence par une vue générale de ce qui doit être réalisé, suivie par l'élaboration d'une liste de caractéristiques prioritaires (carnet de commandes) que le propriétaire du produit souhaite réaliser.

La Figure 1.2 montre les rôles, les informations et les processus qui composent la méthode Scrum [1].

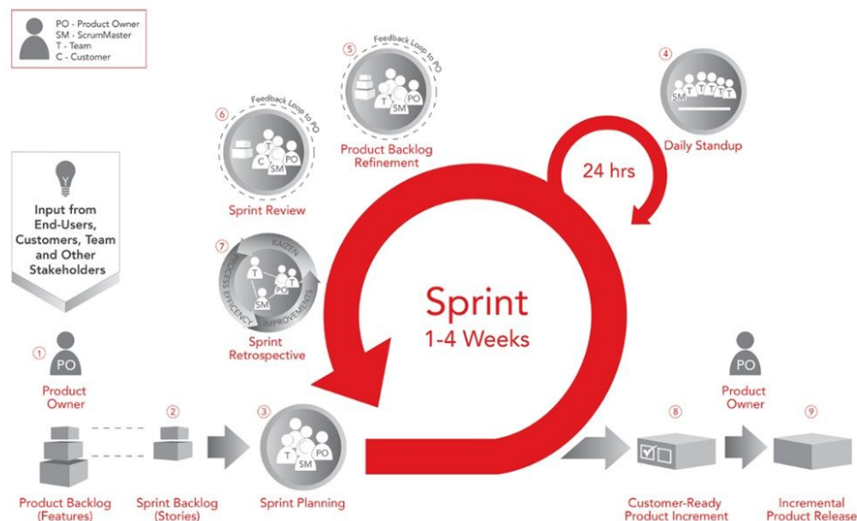


Figure 1.2. Le processus de Scrum

Dans Scrum, l'équipe se concentre sur la réalisation d'un logiciel de qualité. Le propriétaire d'un projet Scrum se focalise sur la définition des caractéristiques que le produit doit avoir (ce qu'il faut construire, ce qu'il ne faut pas construire et dans quel ordre) et à dépasser tout obstacle qui risquent de se mettre en travers de la route du produit.

L'équipe Scrum se compose des rôles suivants :

- Le **Scrum Master** agit comme un coordinateur et un coach pour l'équipe Scrum, en assurant qu'elle comprenne et applique les principes et les processus Scrum de manière efficace. Il aide également à résoudre les obstacles et les problèmes rencontrés par l'équipe.
- Le **Product Owner** est chargé de définir la vision du produit, de déterminer les fonctionnalités à développer et de priorité le backlog du produit en fonction de la valeur qu'elles apportent au client (ou à l'entreprise).
- L'**équipe de développement** est autonome et auto-organisée, et elle est composée de professionnels ayant de hauts niveaux d'expertise capables de collaborer pour livrer des résultats de haute qualité.

1.4.2 Planification et déroulement

Le stage, d'une durée de deux mois, a été structuré en cinq sprints logiques, chacun se concentrant sur un objectif majeur. Cette planification a permis d'assurer une progression constante, de la conception à la réalisation finale.

- **Sprint 1 : Analyse et Cadrage** : Compréhension des exigences, étude des services AWS pertinents et des meilleures pratiques Terraform.
- **Sprint 2 : Conception de l'Architecture** : Création des diagrammes d'architecture, conception du réseau (VPC), des rôles IAM et de la structure des modules Terraform.
- **Sprint 3 : Développement des Modules d'Infrastructure** : Ce sprint constitue le cœur technique du projet. Il est dédié à la matérialisation de l'architecture cloud conçue précédemment en code, via le développement et le déploiement effectif des composants d'infrastructure.

- **Sprint 4 : Implémentation de la CI/CD** : Développement du workflow GitHub Actions, tests de la pipeline et déploiement de l'environnement de développement.
- **Sprint 5 : Documentation et Finalisation** Rédaction du rapport, documentation du code et préparation de la soutenance.

Le diagramme de Gantt ci-dessous (Figure 1.3) illustre visuellement le déroulement de ces phases tout au long du projet.

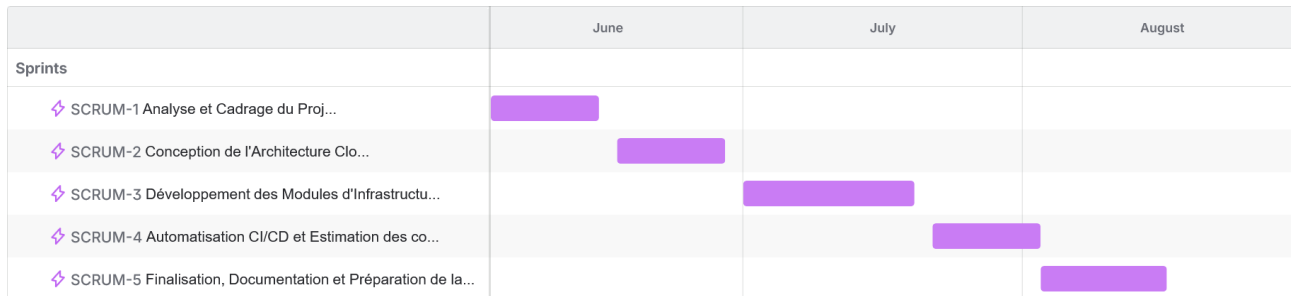


Figure 1.3. Diagramme de Gantt du projet

1.5 Conclusion

Ce premier chapitre a posé le cadre du projet, en présentant l'organisme d'accueil, la problématique de la gestion manuelle de l'infrastructure et la solution automatisée proposée. Il a également détaillé la méthodologie de gestion de projet qui a structuré sa réalisation. Le chapitre suivant explorera les fondements théoriques sur lesquels repose notre travail.

Revue de Littérature : Fondements Théoriques

2.1 Introduction

Avant de détailler les spécifications techniques de notre solution, il est essentiel de définir les concepts fondamentaux qui sous-tendent notre approche. Ce chapitre explore les piliers théoriques de notre projet : l'Infrastructure as Code (IaC), la philosophie DevOps, les principes du Cloud-Native et l'architecture des microservices.

2.2 L'Infrastructure as Code (IaC)

L'Infrastructure as Code (IaC) est une pratique qui consiste à gérer et provisionner l'infrastructure informatique (réseaux, serveurs, bases de données) au moyen de fichiers de définition lisibles par machine, plutôt que par une configuration manuelle [2].

2.2.1 Approche Déclarative vs. Impérative

Terraform se distingue par son approche **déclarative**. Cela signifie que l'on décrit dans le fichier de configuration l'état final souhaité de l'infrastructure par exemple, « 5 serveurs avec telle configuration réseau » et c'est Terraform qui détermine automatiquement les actions à réaliser pour atteindre cet état.

En opposition, l'approche **impérative** consiste à écrire un script qui détaille explicitement chaque étape à exécuter, dans un ordre précis (créer le réseau, puis chaque serveur, etc.). Cette méthode demande une gestion fine des opérations et peut être plus sujette aux erreurs.

La Figure 2.1 illustre ce processus séquentiel où chaque commande doit être spécifiée.

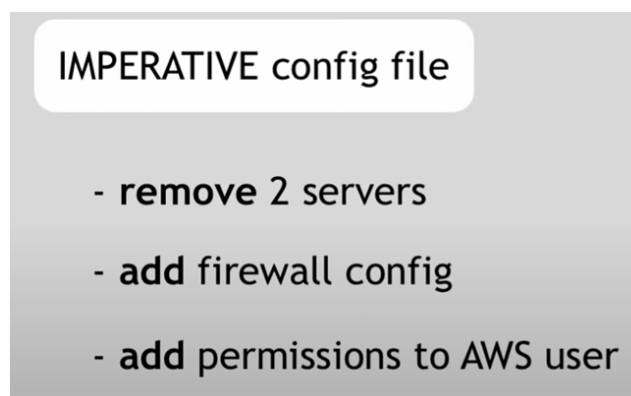


Figure 2.1. Approche Impérative

L'approche déclarative simplifie la gestion des modifications : pour ajouter ou retirer des ressources, il suffit de mettre à jour l'état désiré dans le fichier. Comme le montre la Figure 2.2, Terraform calcule ensuite lui-même les actions nécessaires pour passer de l'état actuel à ce nouvel état. Cela rend le code plus lisible, facilite la maintenance et réduit les risques d'erreurs.

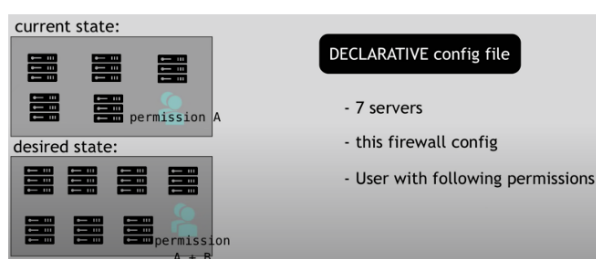


Figure 2.2. Approche Déclarative

2.3 La Philosophie DevOps

Le DevOps est une culture et un ensemble de pratiques qui visent à unifier le développement logiciel (Dev) et l'administration des systèmes (Ops). L'objectif est de raccourcir le cycle de vie du développement et de fournir une livraison continue de haute qualité. L'automatisation est au cœur de cette philosophie. Des pratiques comme l'intégration et le déploiement continus (CI/CD) et l'Infrastructure as Code sont des piliers fondamentaux du DevOps, car elles permettent de créer des processus de livraison rapides, fiables et reproductibles. Notre projet s'inscrit directement dans cette démarche en automatisant entièrement le provisionnement de l'infrastructure.

La Figure 2.3 illustre la méthodologie DevOps et ces étapes principaux.

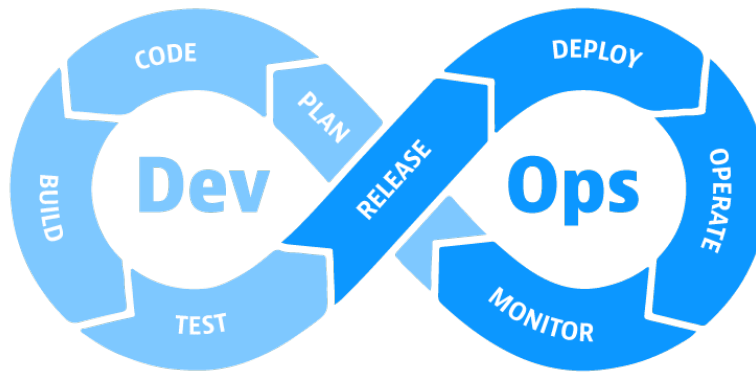


Figure 2.3. Schéma de DevOps

2.4 Les Principes du Cloud-Native

Une application "Cloud-Native" est conçue spécifiquement pour tirer parti des avantages du cloud computing. Il ne s'agit pas seulement de faire tourner une application sur le cloud, mais de l'architecturer pour qu'elle soit résiliente, scalable et flexible. Les principes clés incluent :

- **La conteneurisation** : Empaqueter les applications et leurs dépendances dans des conteneurs (comme Docker) pour assurer la cohérence entre les environnements.
- **L'orchestration** : Utiliser des outils (comme Amazon ECS dans notre cas) pour gérer automatiquement le déploiement, la mise à l'échelle et le fonctionnement des conteneurs.
- **L'architecture microservices** : Décomposer les applications en petits services indépendants, comme nous le verrons dans la section suivante.

La solution proposée constitue un socle adapté pour héberger des applications respectant ces principes.

La Figure 2.4 illustre les principes du cloud native tels que la fusion entre : CI/CD, Conteneurs, Microservices, etc.

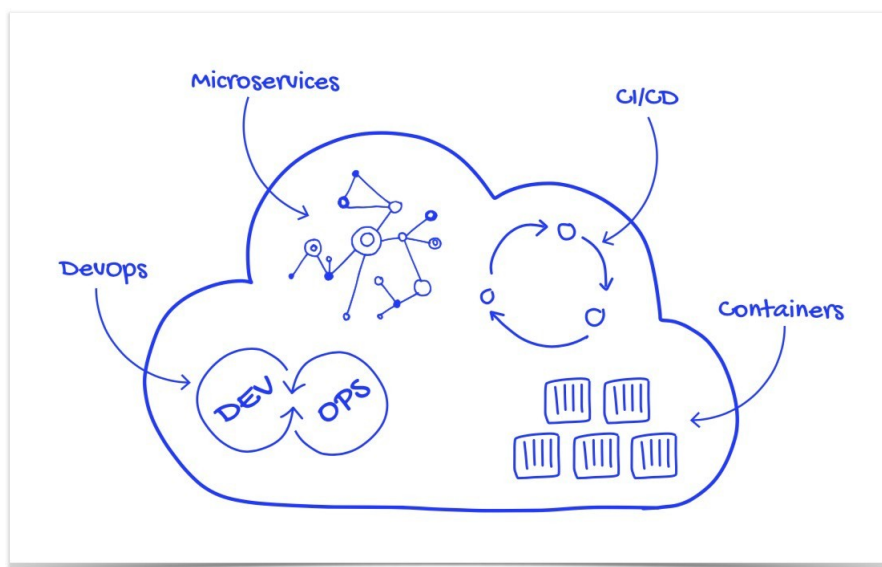


Figure 2.4. Principes du cloud native

2.5 Architecture Applicative Cible : Les Microservices

Bien que ce projet se concentre sur l'infrastructure, il est conçu pour supporter une architecture applicative spécifique : les microservices. Cette section décrit cette architecture et comment notre infrastructure est optimisée pour elle.

L'architecture de microservices divise une application en services indépendants qui communiquent via des API. L'infrastructure mise en place permet d'héberger de tels services, en offrant la possibilité à chacun d'être déployé et mis à l'échelle de manière autonome. Cette approche, adoptée par des entreprises comme Netflix, est idéale pour les environnements cloud modernes.

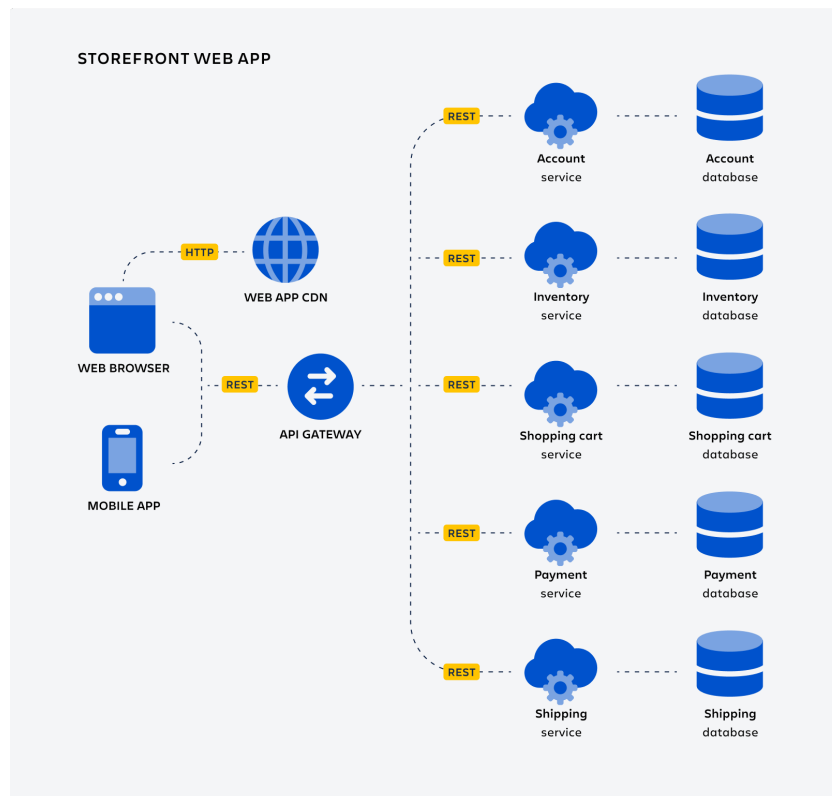


Figure 2.5. L'architecture de microservices

2.5.1 Réponse aux besoins des microservices

L'infrastructure conçue répond aux exigences clés des microservices :

- **Déploiement indépendant** : Grâce à Amazon ECS et à notre configuration déclarative, chaque microservice peut être déployé et mis à jour sans impacter les autres.
- **Autonomie des équipes** : L'approche "self-service" via les fichiers `.tfvars` permet à chaque équipe de gérer le cycle de vie de son propre microservice sans dépendre d'une équipe d'infrastructure centrale.
- **Scalabilité granulaire** : Chaque service peut être mis à l'échelle indépendamment des autres en ajustant simplement son `desired_count`, optimisant ainsi l'utilisation des ressources.
- **Résilience** : En isolant les services, la défaillance de l'un n'entraîne pas la chute de toute l'application. Notre infrastructure Multi-AZ renforce encore cette résilience.

2.6 Conclusion

Ce chapitre a présenté les fondements théoriques de notre projet : l'Infrastructure as Code pour l'automatisation et la fiabilité, la philosophie DevOps pour l'efficacité et la collaboration, les principes Cloud-Native pour la résilience et la scalabilité, ainsi que l'architecture microservices comme modèle applicatif cible. Ces éléments constituent la base sur laquelle repose la suite de notre travail.

Spécification Des Exigences

3.1 Introduction

Ce chapitre définit le cahier des charges de notre projet. Il détaille les exigences fonctionnelles et non fonctionnelles de l'infrastructure à construire, analyse les solutions alternatives et présente la conception architecturale retenue, à la fois sur le plan logique et physique.

3.2 Étude des exigences

3.2.1 Objectifs du Projet

L'objectif principal est de construire une **fondation d'infrastructure cloud** sur AWS, entièrement automatisée pour les équipes de développement. Ce projet vise à fournir un cadre de travail (*framework*) défini par le code (IaC), qui standardise la manière dont les applications sont provisionnées, configurées et gérées. L'enjeu est de garantir la sécurité, la performance et la cohérence entre tous les environnements de développement, en offrant aux développeurs un processus de déploiement simple et fiable.

3.2.2 Besoins fonctionnels (Capacités de la plateforme)

Les exigences fonctionnelles décrivent ce que la plateforme doit être capable de faire.

- **Gestion des environnements**

- La plateforme doit pouvoir déployer des environnements multiples et isolés (ex: develop, uat, prod).
- Chaque environnement doit avoir sa propre configuration réseau (VPC, subnets) et ses propres ressources.
- La configuration de chaque environnement doit être gérée via des fichiers de variables dédiés (.tfvars).

- **Déploiement de services en self-service**

- Les développeurs doivent pouvoir déployer un nouveau microservice en l'ajoutant à une liste de configuration.

- La plateforme doit automatiquement provisionner les ressources ECS (Task Definition, Service) pour ce service.
 - La plateforme doit configurer le routage réseau (via l'ALB) pour rendre le service accessible.
 - Les développeurs doivent pouvoir spécifier les ressources nécessaires (CPU, mémoire) et le nombre d'instances ('desired-count') pour chaque service.
- **Gestion de la configuration et des secrets**
 - La plateforme doit permettre d'injecter des variables d'environnement dans les conteneurs.
 - Elle doit fournir un moyen sécurisé de gérer les secrets (clés d'API, mots de passe de base de données), par exemple via AWS Secrets Manager.
- **Automatisation (CI/CD)**
 - Toute modification du code de l'infrastructure doit déclencher une pipeline automatisée.
 - La pipeline doit valider la syntaxe et le format du code Terraform ('fmt', 'validate').
 - La pipeline doit générer un plan d'exécution ('terraform plan') pour montrer les changements avant toute application.
- **Observabilité (Logging Monitoring)**
 - La plateforme doit automatiquement créer un groupe de logs CloudWatch pour chaque service déployé.
 - Les logs de tous les conteneurs doivent être centralisés et accessibles.
 - Des métriques de base (CPU, mémoire) doivent être disponibles pour chaque service.

3.2.3 Besoins non fonctionnels (Attributs de qualité de la plateforme)

- **Sécurité**
 - Le principe du moindre privilège doit être appliqué à tous les rôles IAM.
 - Les données sensibles (ex: bases de données) doivent être dans des sous-réseaux privés, inaccessibles depuis Internet.
 - Le trafic entrant doit être filtré par un Web Application Firewall (WAF) dans les environnements de production.
 - La gestion des secrets doit être centralisée et sécurisée.
- **Fiabilité et Haute Disponibilité**
 - L'infrastructure doit être déployée sur au moins deux Zones de Disponibilité (Multi-AZ) pour tolérer la panne d'un datacenter.
 - Les services doivent avoir un 'desired-count' d'au moins 2 en production pour assurer la continuité.
 - Les bases de données en production doivent être configurées en mode Multi-AZ.
- **Performance et Scalabilité**

- La plateforme doit être capable de supporter l’ajout de nouveaux services sans dégradation des performances.
 - L’utilisation des ressources (CPU/Mémoire) doit être optimisée pour le coût.
 - La plateforme devrait pouvoir intégrer des politiques d’auto-scaling pour ajuster le nombre de conteneurs en fonction de la charge.
- **Maintenabilité et Évolutivité**
 - L’infrastructure doit être définie dans des modules Terraform réutilisables et bien documentés.
 - L’ajout d’un nouveau microservice ne doit pas nécessiter de modification du code des modules, seulement des fichiers de configuration.

3.3 Exigences Techniques

Pour la mise en œuvre du projet, nous utiliserons les outils suivants :

- **Amazon Web Services (AWS)** est une plateforme de services cloud offrant une large gamme de solutions comme le calcul, le stockage, les bases de données et le réseau. Elle permet de déployer des infrastructures scalables, résilientes et sécurisées à grande échelle. AWS est un choix de référence pour les entreprises modernes souhaitant héberger leurs applications dans le cloud. La Figure 3.1 montre le logo d’AWS [3].



Figure 3.1. Logo d’AWS

- **Terraform** est un outil d’Infrastructure as Code (IaC) développé par HashiCorp, permettant de décrire l’infrastructure souhaitée via des fichiers de configuration. Il facilite le déploiement, la mise à jour et la gestion des ressources cloud de manière déclarative et automatisée. Grâce à Terraform, l’infrastructure devient versionnable, reproductible et auditable. La Figure 3.2 montre le logo de Terraform [4].



Figure 3.2. Logo de Terraform

- **Infracost** est un outil open-source qui permet d'estimer en temps réel le coût des changements d'infrastructure décrits avec Terraform. Il s'intègre dans les pipelines CI/CD pour aider les équipes à prendre des décisions éclairées en matière de coût avant chaque déploiement. La Figure 5.11 montre le logo d'Infracost [5].



Figure 3.3. Logo d'Infracost

- **GitHub Actions** est un service d'intégration et de livraison continues (CI/CD) proposé par GitHub. Il permet d'automatiser les workflows de développement comme les tests, le déploiement ou l'analyse de code, directement à partir des événements du dépôt Git. C'est un outil essentiel pour garantir des livraisons rapides et fiables. La Figure 3.4 montre le logo de GitHub Actions [6].



Figure 3.4. Logo de GitHub Actions

3.4 Analyse de l'existant et des solutions alternatives

L'analyse de l'existant est une étape cruciale pour positionner notre projet. L'objectif n'est pas de comparer des applications CRM/ERP, mais d'évaluer les différentes approches et plateformes

permettant de déployer et gérer une infrastructure cloud. Cette analyse nous permet de justifier nos choix technologiques.

Voici les principales alternatives à notre approche personnalisée avec Terraform et GitHub Actions :

- **AWS Elastic Beanstalk** : C'est une plateforme en tant que service (PaaS) d'AWS qui abstrait une grande partie de la gestion de l'infrastructure. L'utilisateur télécharge son code, et Beanstalk s'occupe de provisionner les serveurs, les load balancers et la mise à l'échelle. C'est plus simple à démarrer, mais offre beaucoup moins de contrôle et de flexibilité sur l'architecture réseau et la sécurité que notre solution basée sur Terraform. Figure 3.5 montre le logo d'AWS Elastic Beanstalk.



Figure 3.5. AWS Elastic Beanstalk Logo

- **HashiCorp Terraform Cloud** : C'est une version gérée de Terraform qui offre des fonctionnalités de collaboration, de gouvernance et de gestion d'état à distance. Elle représente une alternative plus mature à notre pipeline CI/CD personnalisée sur GitHub Actions. Bien que très puissante, sa mise en place peut être plus complexe et coûteuse pour démarrer un projet. Notre solution avec GitHub Actions est plus légère et directement intégrée à notre gestion de code source. Figure 3.6 montre le logo de Terraform Cloud.



Figure 3.6. Terraform Cloud Logo

- **AWS CloudFormation** : C'est le service d'Infrastructure as Code natif d'AWS. Il permet, comme Terraform, de définir son infrastructure dans des fichiers (JSON ou YAML). Cependant, CloudFormation est spécifique à AWS, tandis que Terraform est agnostique et peut gérer plusieurs fournisseurs de cloud. Terraform est aussi souvent considéré comme ayant une syntaxe plus lisible et une communauté plus large. Figure 3.7 montre le logo d'AWS CloudFormation.



AWS CloudFormation

Figure 3.7. AWS CloudFormation Logo

3.5 Conclusion

Ce chapitre a établi un cahier des charges précis et une conception architecturale claire. En partant des exigences fonctionnelles et non fonctionnelles, nous avons justifié nos choix technologiques et modélisé une solution robuste qui servira de plan directeur pour la réalisation technique.

Analyse et Conception

4.1 Introduction

Ce chapitre se concentre sur la conception et l'analyse, deux étapes clés dans la mise en place de systèmes d'information de type interactif. L'objectif est de mieux comprendre la structure du projet et de garantir son succès.

4.2 Méthode de modélisation

Pour la conception de notre infrastructure, nous avons adopté une approche de modélisation moderne, entièrement basée sur les principes de l'Infrastructure as Code (IaC) et les pratiques DevOps. Plutôt que d'utiliser des méthodes de modélisation traditionnelles comme MERISE, qui sont axées sur les données et les traitements applicatifs, notre démarche s'est concentrée sur la définition de l'infrastructure elle-même comme un système logiciel.

Cette approche repose sur trois piliers fondamentaux :

- **Reproductibilité et Automatisation** : En utilisant **Terraform**, nous avons modélisé l'ensemble de notre infrastructure cloud sous forme de code versionné. Cela garantit que chaque composant (réseau, serveurs, bases de données) est défini de manière explicite et peut être recréé de façon identique et automatisée, éliminant ainsi les erreurs manuelles.
- **Conception Modulaire et Cloud-Native** : L'infrastructure a été décomposée en modules Terraform réutilisables (par exemple, un module pour le réseau, un pour les services conteneurisés). Cette conception modulaire, alignée avec les architectures cloud-native, offre une grande flexibilité et facilite la maintenance et l'évolution du système.
- **Intégration Continue (CI/CD)** : Notre modèle de développement intègre une pipeline CI/CD avec **GitHub Actions**. Chaque modification apportée au code de l'infrastructure est automatiquement validée et planifiée, ce qui assure un cycle de vie contrôlé et agile, en parfaite adéquation avec la méthodologie Scrum décrite précédemment.

La Figure 4.1 illustre ce cycle de modélisation et de livraison continue, où l'infrastructure est traitée comme du code, depuis sa définition jusqu'à son déploiement automatisé.

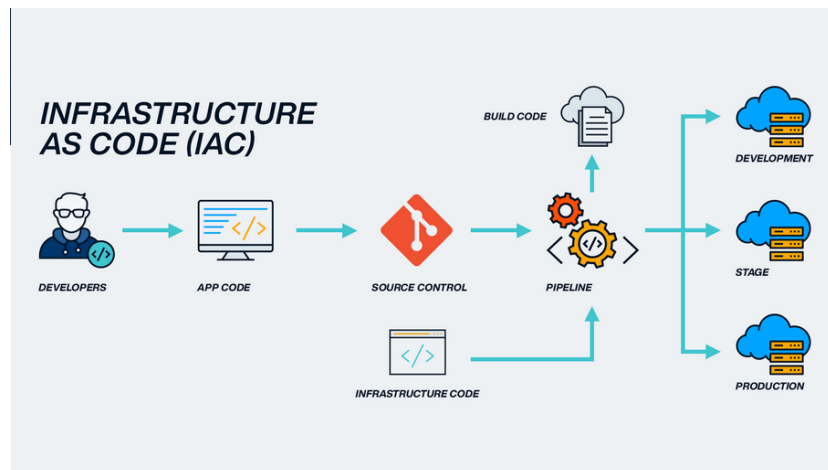


Figure 4.1. Cycle de modélisation et de livraison continue avec IaC et DevOps

4.3 Identification des acteurs de la plateforme

Lors de la phase d'analyse, nous avons déterminé les différents types d'utilisateurs qui interagissent avec la plateforme de déploiement interne. Chaque acteur a des responsabilités et des permissions distinctes.

Le tableau 4.1 représente les acteurs de notre plateforme.

Acteurs	Rôles et responsabilités
Développeur d'application	Définit les besoins de son application (CPU, mémoire, variables d'environnement) via les fichiers <code>.tfvars</code>. Déploie son code dans un registre de conteneurs (ECR). Consulte les logs dans CloudWatch pour le débogage. N'a pas besoin de connaître l'infrastructure sous-jacente.
Ingénieur DevOps / Cloud (Mon rôle)	Conçoit et maintient les modules Terraform de la plateforme. Gère la CI/CD pour automatiser et sécuriser les déploiements. Surveille la santé, la sécurité et les coûts sur AWS. Approuve les changements proposés par les développeurs.
Responsable de Sécurité	Audite les rôles IAM, pare-feux et configurations sensibles. Définit les politiques de sécurité à appliquer. Analyse les journaux du WAF et d'audit.

Tableau 4.1. Acteurs de la plateforme et leurs rôles

4.4 Conception de l'Architecture

La phase de conception est essentielle pour garantir la construction d'une infrastructure cloud robuste, sécurisée et évolutive. Dans les sections suivantes, nous allons présenter la conception de notre plateforme à travers deux diagrammes clés qui illustrent sa structure et son fonctionnement.

4.4.1 Diagramme d'Architecture Logique

Ce diagramme présente une vue conceptuelle de la plateforme, organisée en couches fonctionnelles. Il montre comment une requête d'un utilisateur final traverse les différentes couches de sécurité, de routage, de répartition de charge et de calcul avant d'atteindre les services applicatifs. Il met également en évidence les interactions entre les services, les bases de données et le registre de conteneurs.

La Figure 4.2 représente l'architecture logique de notre plateforme.

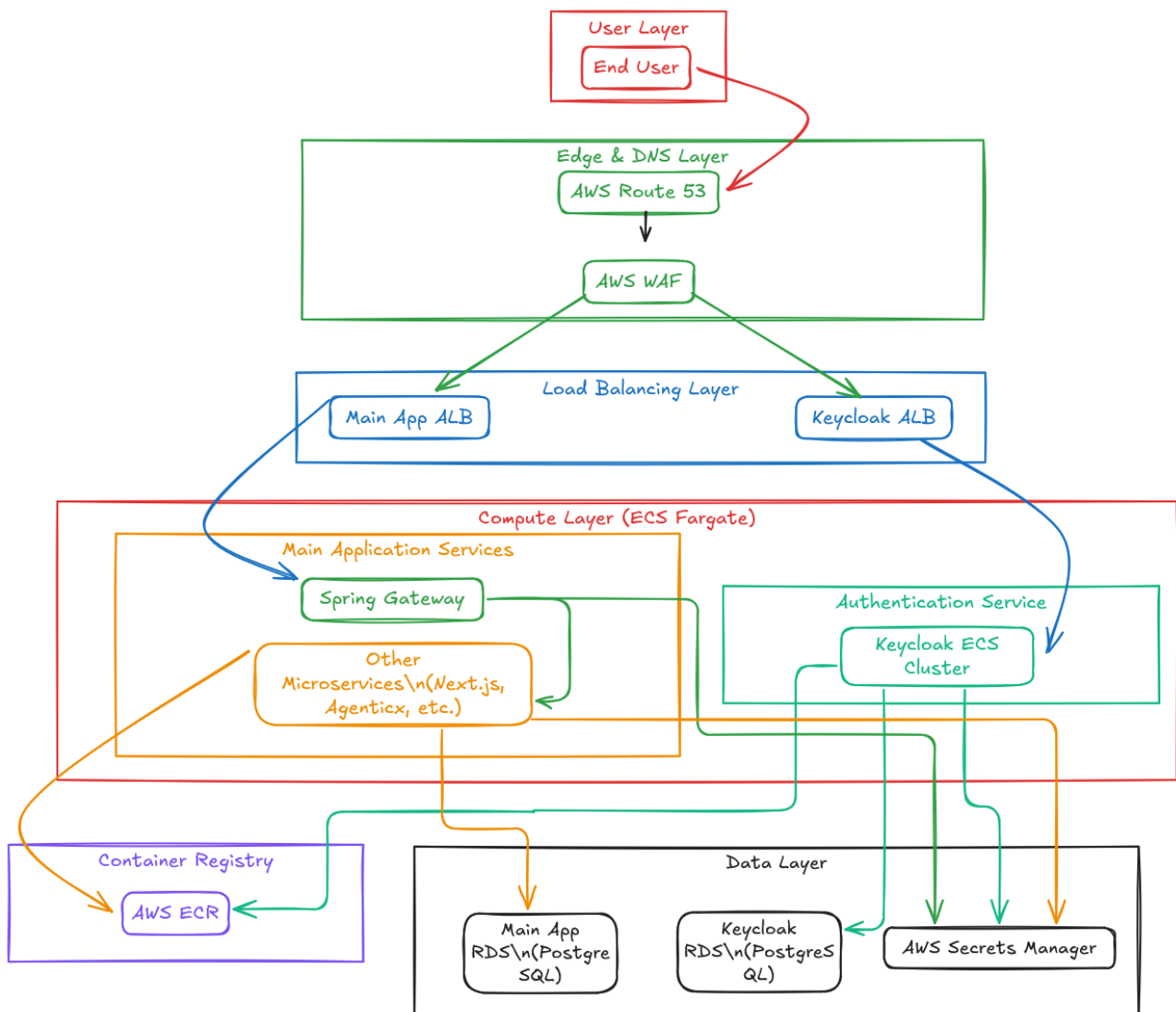


Figure 4.2. Diagramme d'Architecture Logique

4.4.2 Diagramme d'Architecture Physique

Ce diagramme détaille l'implémentation concrète de notre architecture sur AWS pour un environnement spécifique (par exemple, développement). Il montre la topologie du réseau VPC, la répartition des ressources dans les sous-réseaux publics et privés, et le déploiement des clusters ECS, des Application Load Balancers et des bases de données. Ce schéma illustre la mise en pratique des principes de haute disponibilité et de sécurité.

La Figure 4.3 représente l'architecture physique de l'environnement de développement.

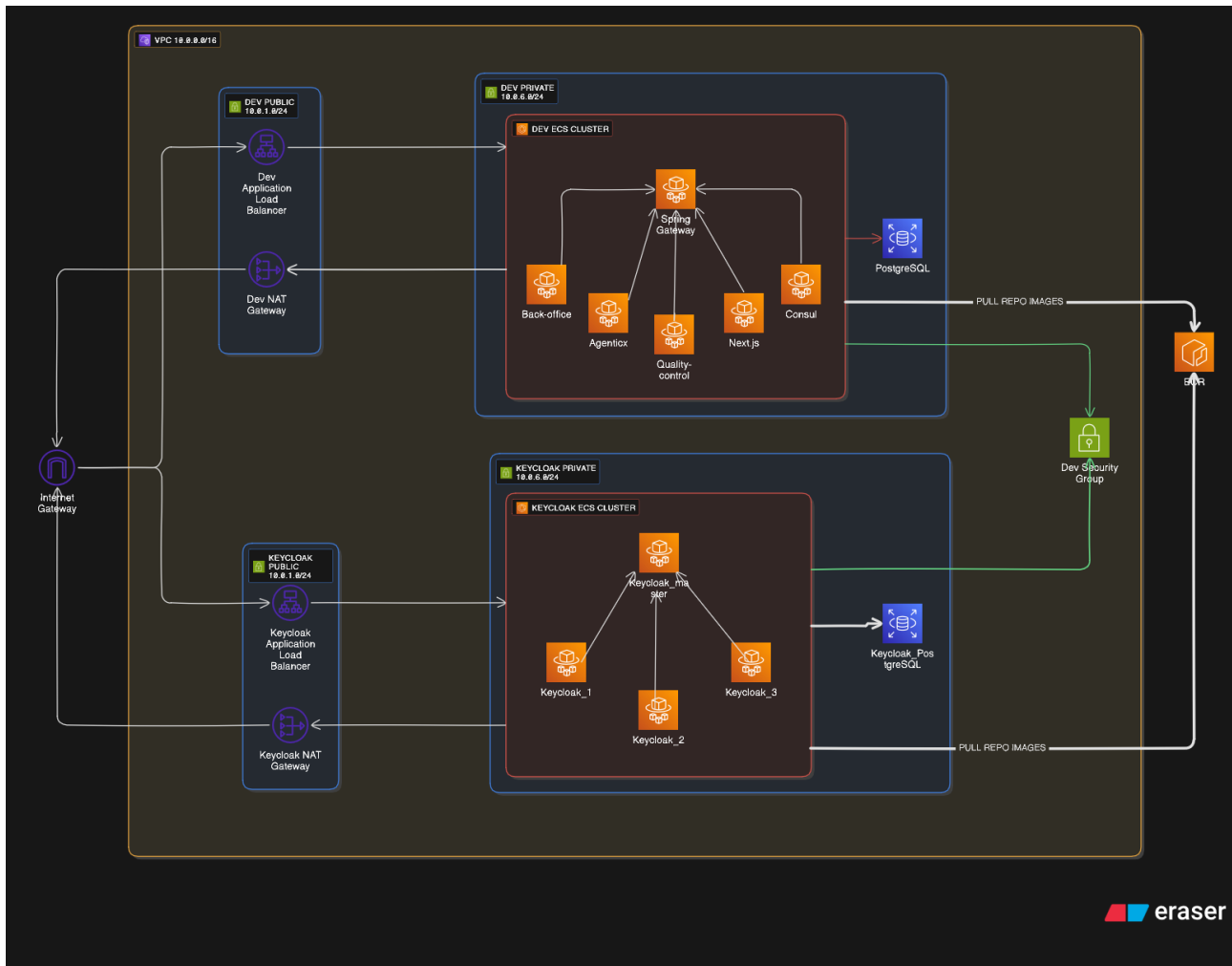


Figure 4.3. Diagramme d'Architecture Physique

4.5 Conclusion

Dans ce chapitre, nous avons détaillé la conception de notre plateforme cloud à travers ses architectures logique et physique. Ces diagrammes illustrent comment les exigences de sécurité, de haute disponibilité et de modularité ont été traduites en une solution technique concrète. Cette conception a servi de plan directeur pour l'implémentation de l'infrastructure avec Terraform.

Réalisation Technique

5.1 Introduction

Après avoir défini les fondements théoriques et les exigences du projet, ce chapitre se concentre sur sa mise en œuvre concrète. Nous commencerons par détailler le fonctionnement interne de Terraform, notre outil principal, avant de présenter l'écosystème technologique complet, la structure du code, et de démontrer le workflow de déploiement automatisé.

5.2 Principes de Fonctionnement de Terraform

Pour bien comprendre la mise en œuvre, il est utile de connaître l'architecture de Terraform. Son fonctionnement repose sur deux composantes essentielles : le **Cœur (Core)** et les **Fournisseurs (Providers)**.

- **Le Cœur de Terraform** est responsable de l'analyse des fichiers de configuration `.tf` que nous écrivons. Il compare l'état désiré décrit dans ces fichiers avec l'état actuel de l'infrastructure, qui est sauvegardé dans un fichier d'état (`terraform.tfstate`). En se basant sur cette comparaison, il génère un plan d'exécution détaillé.
- **Les Fournisseurs (Providers)** sont des plugins qui agissent comme des traducteurs. Chaque fournisseur (par exemple, pour AWS, Azure, ou Google Cloud) sait comment communiquer avec l'API de sa technologie respective. Le Cœur délègue le plan d'exécution au fournisseur approprié, qui se charge alors d'effectuer les appels d'API nécessaires pour créer, modifier ou supprimer les ressources.

Ce duo Cœur/Fournisseur est ce qui permet à Terraform d'être à la fois puissant et extrêmement flexible, capable de gérer une multitude de services cloud et d'outils.

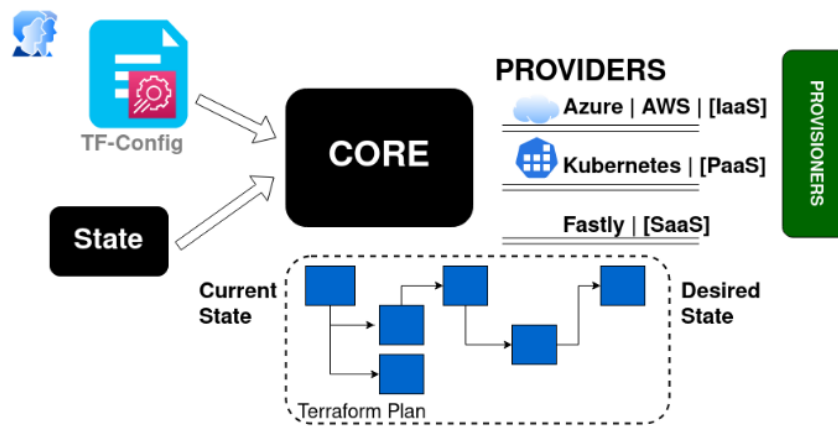
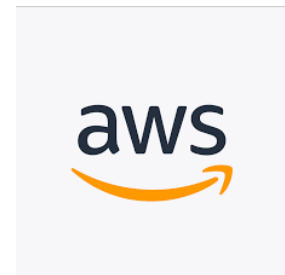


Figure 5.1. Architecture interne de Terraform

5.3 Environnement de développement technique

- **Amazon Web Services (AWS)** est la plateforme de cloud computing leader du marché, offrant une gamme complète de services d'infrastructure. Pour notre projet, nous utilisons des services fondamentaux comme **VPC** pour l'isolation réseau, **ECS avec Fargate** pour l'orchestration de conteneurs sans serveur, **RDS** pour les bases de données gérées, et **IAM** pour la gestion sécurisée des accès. AWS constitue la fondation sur laquelle toute notre infrastructure est construite.



- **Terraform** est un outil d'Infrastructure as Code (IaC) open-source créé par HashiCorp. Il nous permet de définir et de provisionner notre infrastructure cloud à l'aide d'un langage de configuration déclaratif. En décrivant nos ressources dans des fichiers de code, nous pouvons automatiser, versionner et gérer notre infrastructure de manière fiable et reproductible, éliminant ainsi les configurations manuelles et les erreurs associées.



- **GitHub Actions** est une plateforme d'intégration et de déploiement continu (CI/CD) intégrée à GitHub. Nous l'utilisons pour automatiser notre flux de travail Terraform. Chaque modification du code de notre infrastructure déclenche automatiquement une série d'actions : validation du code, initialisation de Terraform, et création d'un plan d'exécution. Cela garantit que chaque changement est vérifié et planifié avant d'être appliqué.



- **Infracost** est un outil qui s'intègre à notre pipeline CI/CD pour estimer le coût des changements d'infrastructure avant même leur déploiement. En analysant les plans Terraform, Infracost génère un rapport détaillé sur l'impact financier des modifications proposées (ajouts, suppressions, mises à jour de ressources). Cela nous permet de prendre des décisions éclairées, de maîtriser les coûts du cloud et d'éviter les mauvaises surprises sur la facture AWS.



5.4 Environnement de developpement logiciel

Visual Studio Code (VS Code) est un éditeur de code gratuit, léger et puissant, compatible avec Windows, macOS, Linux et Raspberry Pi OS. Il offre un support intégré pour JavaScript, TypeScript et Node.js, ainsi qu'un large choix d'extensions pour d'autres langages (C++, C, Java, Python, PHP, Go, etc.) et environnements (Docker, Kubernetes, .NET, Unity, etc.). VS Code est polyvalent, fonctionnant sur le bureau et sur le web, et est pris en charge par divers clouds tels qu'AWS, Azure et GCP.



5.5 Mise en Œuvre et Démonstration de la Plateforme

5.5.1 Structure du Code et Gestion des Environnements

La réalisation du projet a commencé par la mise en place d'une structure de code Terraform à la fois modulaire et scalable.

Notre projet est organisé autour d'un répertoire central `modules/` qui contient des briques de construction indépendantes pour chaque service AWS (par exemple, un module pour le réseau, un pour ECS, un pour RDS, etc.). Cette approche modulaire est une bonne pratique qui facilite la réutilisabilité du code, simplifie la maintenance et permet de faire évoluer l'infrastructure de manière contrôlée. Chaque module est autonome, avec ses propres variables d'entrée et de sortie.

La configuration spécifique à chaque environnement de déploiement (`develop`, `uat`, `prod`) est gérée dans le répertoire `live/`. Chaque environnement possède son propre fichier `.tfvars`, qui agit comme un plan de déploiement en définissant les modules à utiliser et en leur passant les variables nécessaires (comme la taille des serveurs ou le nombre de conteneurs). Cette séparation stricte entre la logique réutilisable (modules) et la configuration spécifique (live) garantit l'isolation et la reproductibilité des environnements.

Les Figures 5.2 et 5.3 présentent respectivement la structure du dossier `modules/` et celle du dossier `live/`.

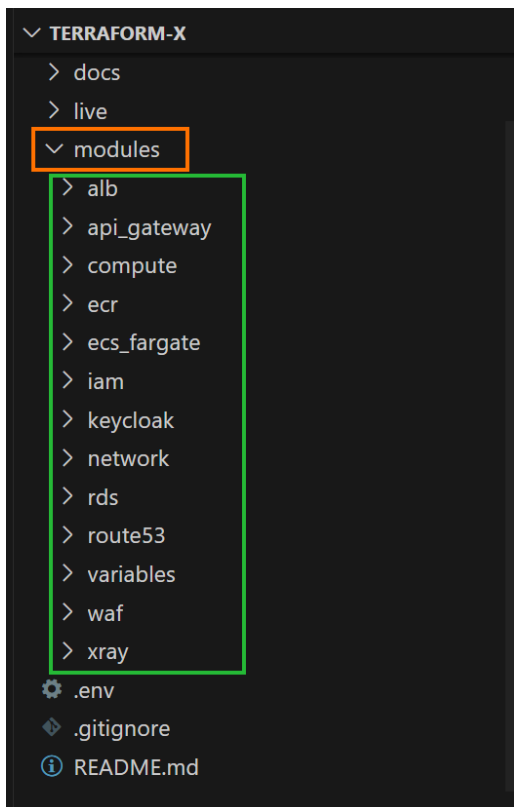


Figure 5.2. Dossier Modules

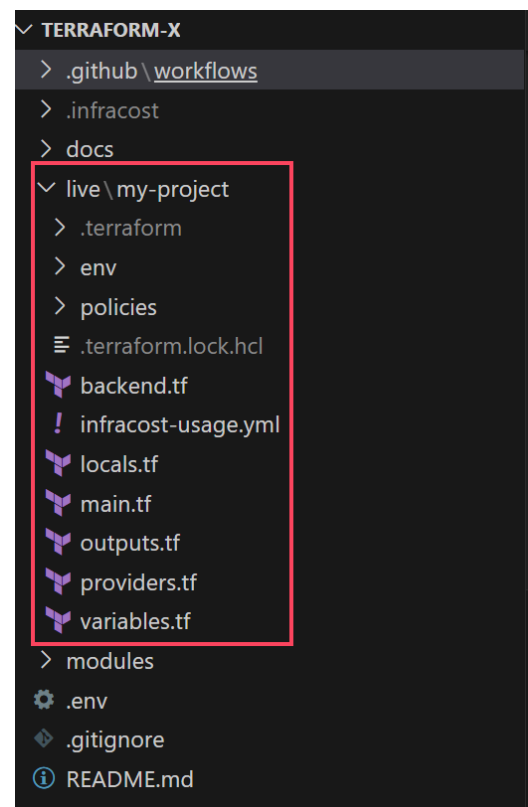


Figure 5.3. Dossier Live

Figure 5.4. Organisation des dossiers Modules et Live

Pour gérer l'état de ces différents environnements, Terraform utilise la notion de **workspace**. Chaque workspace (develop, uat, prod) maintient un fichier d'état distinct, ce qui permet de gérer plusieurs déploiements à partir de la même base de code sans risque d'interférence. La Figure 5.5 montre les workspaces définis pour notre projet.

```
PS D:\PC\Projects\Stage_2\terraform-x\live\my-project> terraform workspace
list
default
* develop
preprod
prod
uat
```

Figure 5.5. Les environnements existants définis via les Workspaces Terraform

Par exemple, pour déployer un service nommé `agenticx` dans l'environnement de développement, il suffit de le déclarer dans le fichier `develop.tfvars` comme suit :

Extrait du fichier `develop.tfvars`

```
ecs_services = {  
  agentix = {  
    name           = "agentix"  
    cpu            = 256  
    memory         = 512  
    desired_count  = 1  
    port           = 8070  
    lifecycle_policy_path = "policies/agentix-lifecycle.json"  
  }  
}
```

5.6 Démonstration du Workflow de Déploiement

C'est le cœur de notre réalisation. Nous allons simuler le déploiement d'un nouveau microservice par un développeur pour démontrer la puissance et la simplicité de la plateforme.

5.6.1 Étape 1 : Déclaration du Nouveau Service

Le processus de déploiement d'un nouveau service débute par une étape simple et centralisée : la modification du fichier de configuration de l'environnement concerné. Pour ajouter un nouveau microservice, par exemple nommé `notifications`, le développeur n'a besoin que d'une seule action : déclarer ce service dans la carte `ecs_services` du fichier `develop.tfvars`.

Cette approche rend le cycle de vie des services totalement déclaratif. Il n'est pas nécessaire de modifier la logique des modules ni d'écrire des scripts manuels. Cela garantit cohérence, auditabilité et rapidité de déploiement.

Les Figures 5.6 et 5.7 illustrent cette étape : la Figure 5.6 montre l'état initial du fichier `develop.tfvars`, avant l'ajout du service, tandis que la Figure 5.7 montre le fichier mis à jour, incluant la configuration du service `notifications`.

```

Plan: 125 to add, 0 to change, 0 to destroy.

Changes to Outputs:
+ ecr_repository_urls = {
+   agentix      = (known after apply)
+   analytix     = (known after apply)
+   consul       = (known after apply)
+   nextjs       = (known after apply)
+   spring-gateway = (known after apply)
+   user-management = (known after apply)
+ }
+ ecs_cluster_id = (known after apply)
+ public_subnet_ids = [
+   (known after apply),
+   (known after apply),
+ ]
+ rds_instance_endpoint = (known after apply)
+ rds_instance_port = (known after apply)
+ route53_nameservers = (known after apply)
+ vpc_id = (known after apply)

```

Figure 5.6. Avant l'ajout du service notifications

```

Plan: 131 to add, 0 to change, 0 to destroy.

Changes to Outputs:
+ ecr_repository_urls = {
+   agentix      = (known after apply)
+   analytix     = (known after apply)
+   consul       = (known after apply)
+   nextjs       = (known after apply)
+   notification = (known after apply)
+   spring-gateway = (known after apply)
+   user-management = (known after apply)
+ }
+ ecs_cluster_id = (known after apply)
+ public_subnet_ids = [
+   (known after apply),
+   (known after apply),
+ ]
+ rds_instance_endpoint = (known after apply)
+ rds_instance_port = (known after apply)
+ route53_nameservers = (known after apply)
+ vpc_id = (known after apply)

```

Figure 5.7. Après l'ajout du service notifications

Figure 5.8. Modification du fichier `develop.tfvars` pour intégrer un nouveau service

5.6.2 Étape 2 : Déclenchement de la Pipeline CI/CD

Une fois la modification poussée sur GitHub, notre pipeline CI/CD — définie dans le fichier `.github/workflows/terraform.yml` — se déclenche automatiquement. Elle est configurée pour s'exécuter à chaque push sur la branche `main`, ou manuellement via l'interface GitHub (grâce au mot-clé `workflow_dispatch`).

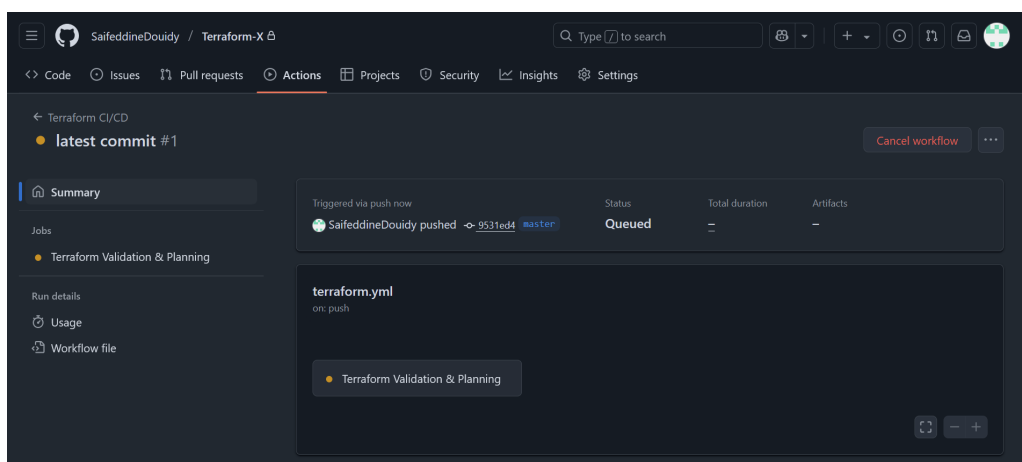


Figure 5.9. Déclenchement automatique de la pipeline CI/CD sur GitHub

5.6.3 Étape 3 : Validation et Planification Automatisées

La pipeline exécute plusieurs étapes critiques sans intervention humaine :

- **Validation** : Le code est vérifié avec `terraform validate` pour s'assurer qu'il est syntaxiquement correct.
- **Planification** : La commande `terraform plan` est exécutée. Cette étape est cruciale car elle ne modifie aucune ressource réelle. Elle agit comme un **aperçu** qui simule les changements et génère un plan d'exécution détaillé. Ce plan permet à l'ingénieur de valider les changements en toute sécurité avant toute application.

- **Analyse des Coûts :** L'outil Infracost analyse le plan généré et publie une estimation de l'impact financier des changements, directement dans la Pull Request pour une visibilité maximale.

La Figure 5.12 illustre les résultats de la planification Terraform ainsi que l'estimation des coûts générée par Infracost.

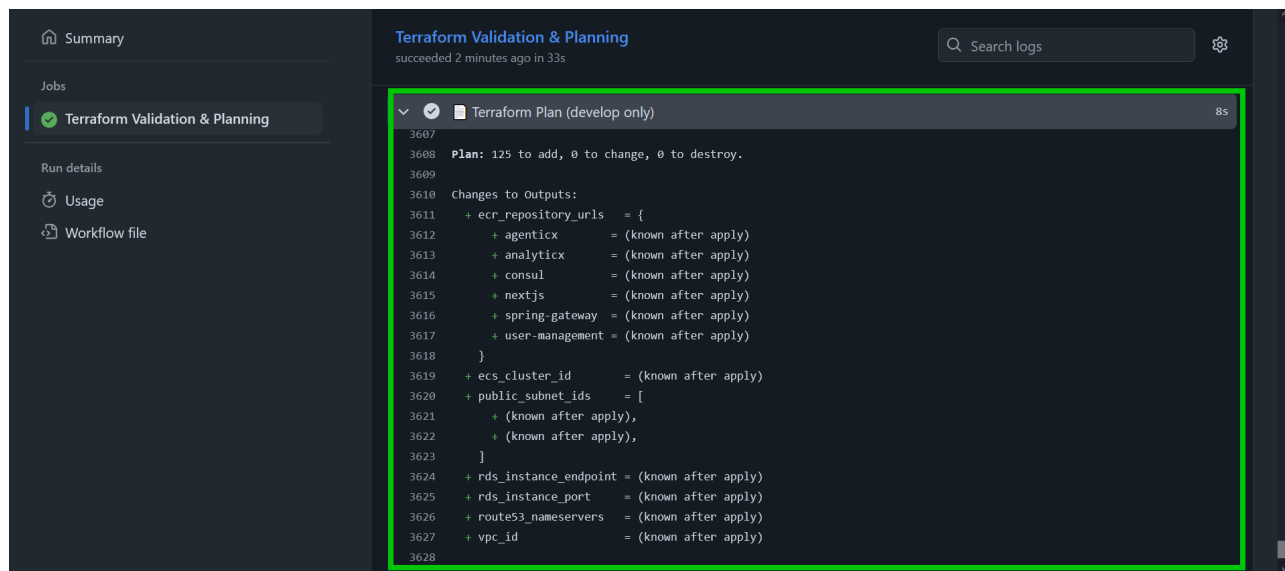


Figure 5.10. Résultat du terraform plan dans GitHub Actions

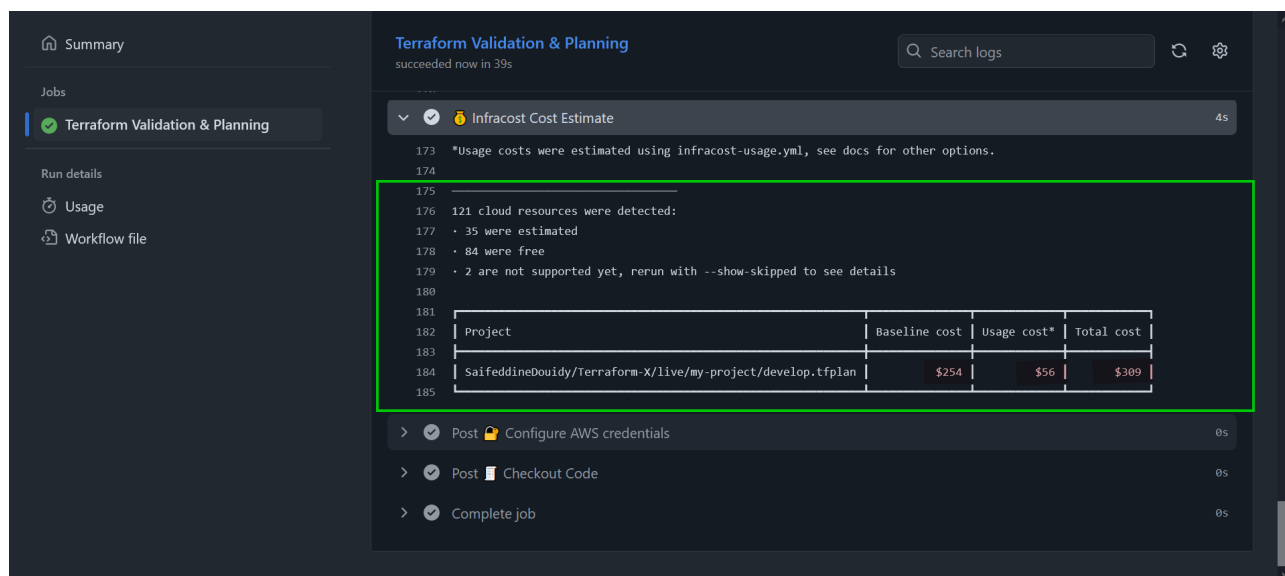


Figure 5.11. Estimation des coûts générée par Infracost

Figure 5.12. Planification Terraform et estimation des coûts via Infracost

5.7 Conclusion

À travers ce processus, nous avons démontré que la plateforme est fonctionnelle. Nous avons transformé une tâche complexe et risquée – le déploiement d'un nouveau service – en un workflow simple, contrôlé et entièrement automatisé. Le "livrable" de ce projet n'est pas une application, mais ce processus robuste et efficace qui constitue la véritable valeur ajoutée pour l'entreprise.

Conclusion Générale et Perspectives

Ce projet de stage a permis de concevoir et de réaliser un **système de déploiement interne** sur Amazon Web Services. En adoptant une approche d'Infrastructure as Code, nous avons remplacé les processus de provisionnement manuels par un mécanisme automatisé, reproductible et sécurisé. L'architecture que nous avons construite fournit une fondation cloud robuste, capable d'accueillir des applications modernes basées sur une architecture de microservices, comme des systèmes ERP et CRM.

Sur le plan technique, ce projet a été une immersion profonde dans les pratiques DevOps et l'écosystème du cloud. Il a permis de maîtriser des outils fondamentaux tels que Terraform pour la gestion déclarative de l'infrastructure, AWS pour ses services cloud, et GitHub Actions pour l'orchestration de pipelines CI/CD. Cette expérience a renforcé des compétences clés en automatisation, en architecture cloud et en gestion de la sécurité et des coûts.

Pour l'avenir, plusieurs améliorations peuvent être envisagées pour enrichir cette solution. Tout d'abord, l'intégration de politiques d'auto-scaling basées sur la charge permettrait d'optimiser dynamiquement les coûts et les performances des services. Ensuite, le déploiement d'outils d'observabilité plus avancés, comme **Prometheus** et **Grafana**, offrirait des tableaux de bord détaillés et des alertes proactives sur la santé de l'infrastructure.

Enfin, la création d'un catalogue de modules Terraform plus étendu (par exemple, pour des bases de données NoSQL ou des services de messagerie comme AWS SQS) pourrait accélérer encore davantage le développement de nouveaux types d'applications.

Ces évolutions visent à renforcer la fiabilité et l'efficacité de ce système, consolidant ainsi son rôle de catalyseur d'innovation pour les équipes de développement de l'entreprise. ““

Webographie

- [1] Scrum. Consulté le 29.04.2024, <https://www.qrpinternational.fr/blog/glossaire/scrum-cest-quoi-definition-scrum/>. En ligne.
- [2] Définition de iac. <https://www.redhat.com/en/topics/automation/what-is-infrastructure-as-code-iac>. En ligne.
- [3] Amazon Web Services. What is aws? <https://aws.amazon.com/what-is-aws/>, 2025.
- [4] HashiCorp. Terraform by hashicorp. <https://www.terraform.io/>, 2025.
- [5] Infracost. Cloud cost estimates for terraform in pull requests. <https://www.infracost.io/>, 2025.
- [6] GitHub. About github actions. <https://docs.github.com/en/actions/learn-github-actions>, 2025.
- [7] Microservices. Consulté le 29.04.2024, <https://www.atlassian.com/fr/microservices/microservices-architecture>. En ligne.

Le présent rapport est une synthèse du travail effectué dans le cadre de mon Stage d'été au sein de la société **ToumAI**. L'objectif de ce projet a été la conception et la mise en œuvre d'une plateforme d'infrastructure cloud automatisée sur AWS.

Mon rôle a couvert l'ensemble du cycle de vie de l'infrastructure, de l'analyse des besoins à la réalisation technique. J'ai conçu l'architecture, défini les ressources avec Terraform et automatisé les déploiements avec une pipeline CI/CD.



Ce projet se concentre sur la création d'une fondation cloud pour des applications comme des ERP/CRM et se divise en quatre étapes principales.

La première étape est l'étude des exigences. Elle inclut l'analyse des besoins fonctionnels et non fonctionnels de la plateforme, comme la sécurité et la haute disponibilité.

La deuxième étape est la revue de littérature. Elle met en avant les bases théoriques du projet, telles que l'Infrastructure as Code (IaC) et la philosophie DevOps. Elle aborde aussi les principes du Cloud-Native et l'architecture microservices comme modèle applicatif cible.

La troisième étape est l'étude technique. Elle couvre le choix de l'architecture cloud sur AWS, des services (ECS, RDS, VPC) et des outils (Terraform, GitHub Actions).

La quatrième étape est la conception modulaire. Utilisant les meilleures pratiques IaC, elle définit les modules Terraform réutilisables pour chaque composant de l'infrastructure.

La cinquième étape est l'implémentation de la pipeline CI/CD, qui assure la validation et la planification automatisée des changements pour garantir des déploiements fiables.

Mots-clés : Cloud, AWS, DevOps, Terraform, IaC, CI/CD, Automatisation.

Sifeddine Doudy

Filière : Ingénierie Informatique et technologies émergentes(2ITE)

3^{ème} année Cycle Ingénieur